Where ARC-AGI-3 Agents Stall: From DQN to a Pre-trained LLM Agent

Musala Ndouvhada[1], **Thendo Nelufule**[1]
Musala Ndouvhada (2686346): 2686346@students.wits.ac.za; Thendo Nelufule (2678518): 2678518@students.wits.ac.za

[1]**School of Computer Science and Applied Mathematics, University of the Witwatersrand, Johannesburg**

Team *Vladmir_Putin*, COMS4061A/7071A Reinforcement Learning, 2026

Abstract

Can agents trained with reinforcement learning learn to explore and solve unseen ARC-AGI-3 games? We trained DQN (with test-time TD learning) and PPO offline on 15 public games and added two improvements to each, every one motivated by a measured failure of the previous stage. All scores below are on the official 0–100 scale: 0.79 means 0.79 out of 100, not 79%. Under a frozen protocol (500 actions per game, 3 training $\times$ 3 evaluation seeds), both baselines are indistinguishable from random on unseen games, and the best learned agent (PPO with RND) scores 0.025 on dev and 0.0005 on held-out games. The diagnostics point to one bottleneck: exploration inside an unseen game, where the useful knowledge is game-specific. Making that knowledge explicit in a training-free graph explorer raises the dev score to 0.79 out of 100. Its transfer is narrow, however. An earlier version drops from 0.50 on dev to 0.13 on held-out games and 0.07 on Kaggle, and the final version scores 0.885 on held-out games but only 0.02 on 15 other games. We then studied the state of the art and adopted the strongest public LLM agent (Franzen's M2 on Tufa Labs' Duck harness) as a research platform. A log audit shows that it, too, loses most of its points to stalls on later levels (43.9 points) rather than to inefficiency (2.6). Our fixes and tests on it (best Kaggle public score 29.15) are all within run-to-run noise, which we quantify. Code: https://github.com/ThendoNelufule/rl-arc-agi3.

1 Introduction

ARC-AGI-3 (ARC Prize Foundation, 2026; Chollet, 2019) consists of turn-based 64×64 games without instructions. An agent must infer controls, goals and rules by acting, and is scored by how efficiently, relative to humans, it completes levels. The Kaggle competition (ARC Prize Foundation and Kaggle, 2026) runs agents offline on 110 hidden games within 9 hours. Rewards are sparse: a random agent completes about one level per 7,500 actions on our training games.

Question. *Can agents trained with RL learn to explore and solve unseen ARC-AGI-3 games, and if not, what limits them?* We answer it in four steps, each motivated by the previous one. (i) DQN and PPO chains with diagnosed failures (§4) show that the bottleneck is exploration inside an unseen game, with game-specific knowledge. (ii) A graph explorer makes that knowledge explicit; it gains strongly on dev and then reveals dev overfitting (§5). (iii) A study of the state of the art (§6) leads us to adopt the strongest public LLM agent as a platform. (iv) On that platform (§7) we test whether the same bottleneck limits a strong pretrained agent. It does: stalls on later levels dominate from DQN to the LLM agent.

Our contributions. The research question and protocol, with a Gymnasium adapter and a frozen train/dev/held-out split; two RL improvement chains and a graph explorer, with every run reported; the platform choice and a reproducible fork; an evaluation proxy and a matched A/B test; a log audit and failure-mode analysis; three fixes (MOD-F) and a scheduling rule (MOD-2) of our own; a verification of retained reasoning; a best-of-N variance analysis; and an investigation of failed Kaggle runs. *Reused and credited:* Franzen's agent (Franzen, 2026), Tufa Labs' harness (Cottaar & Tufa Labs, 2026), the model and serving stack (Qwen Team, 2026; Zheng et al., 2024), and two public add-ons (heliosli, 2026; shiiin9 & Affectify ARC, Osaka Metropolitan University, 2026).

2 Background

Scoring (RHAE; verified against the official scorer, `arc-agi` 0.9.8). Level l, with human baseline b_l and solved in a_l actions, earns $\min(115,\ 100(b_l/a_l)^2)$; unfinished levels earn 0. A game's score is the level-index-weighted mean ($w_l = l$), capped at $100\times$ the weighted share of completed levels, so it is at most 100. The benchmark score is the mean over games, on a 0–100 scale. Late levels dominate. **DQN** (Mnih et al., 2015) minimises the Huber loss to $y = r + \gamma \max_{a'} Q_{\bar{\theta}}(s', a')$ using replay and a target network. **PPO** (Schulman et al., 2017) maximises $L^{\text{CLIP}}(\theta) = \mathbb{E}_t[\min(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1{-}\epsilon, 1{+}\epsilon)\hat{A}_t)]$ with $\rho_t = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ and GAE advantages $\hat{A}_t = \sum_l (\gamma\lambda)^l \delta_{t+l}$, $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ (Schulman et al., 2016), plus a value loss and an entropy bonus. Invalid actions are masked in the softmax (Huang & Ontañón, 2022). **Exploration bonuses:** counts (Bellemare et al., 2016), and RND, the error of a trained predictor against a fixed random network (Burda et al., 2019). **ARC-AGI-3 agents.** *Stochastic Goose* (Smit, 2025) (preview winner) trains a CNN online to predict which actions change the frame. Tufa Labs' *Duck* harness (Cottaar & Tufa Labs, 2026) (Milestone 1) runs an LLM tool agent. Franzen's Milestone-2 solution (Franzen, 2026) patches it with tools, guards and a scheduler over 110 games, served by SGLang (Zheng et al., 2024) with a 4-bit Qwen3.8-Flash-Next (Qwen Team, 2026). OpenAI reported that retaining reasoning between turns, plus compaction, tripled their score (OpenAI, 2026).

3 Formulation and protocol

Each game is an episodic POMDP (hidden step counters, §8), treated as an MDP over frames by our adapter (`rl/arc_gym.py`). **Observation** $o_t \in \{0..15\}^{1\times64\times64}$: the final animation layer as colour ids, one-hot encoded in the network. Raw pixels keep the formulation game-agnostic. **Actions** `Discrete(70)`: ACTION1–5, ACTION7 (undo), and 64 ACTION6 clicks at the centres of an 8×8 grid. A single space with a spatial click head keeps clicks learnable in 400k steps. Illegal actions are no-ops that still cost a step; `action_masks()` exposes legality. Only the graph explorer may also click any pixel ($70{+}64y{+}x$), approved once the grid was shown to be its bottleneck. **Reward** $r_t = \max(0, \Delta\,\text{levels})$, used for training only; we always report the official score. **Termination** on WIN or GAME_OVER; training truncates at 500 actions. In evaluation, GAME_OVER triggers the engine's reset within the same budget. **What persists.** Within a game: DQN-online weights and its replay, novelty counts, the graph and its statistics, and LSTM state. Between levels: weights and statistics persist, but the graph is cleared. Between games and evaluation runs: nothing. Weights are restored and each seed gets a fresh Arcade and scorecard. All learning from other games is *offline* on the 15 training games; *online* adaptation uses only the agent's own interaction. No game-specific code or hidden state is used. **Environments and split.** We used only the 25 public games, offline (`arc-agi` 0.9.8, `arcengine` 0.9.3). They were split once, stratified by type, and frozen (seed 2026): *train* (15: ar25, cn04, ft09, g50t, m0r0, r11l, re86, s5i5, sb26, sc25, sp80, su15, tr87, vc33, wa30), *dev* (5: dc22, ka59, lf52, lp85, ls20; all decisions), and *held-out* (5: bp35, cd82, sk48, tn36, tu93), evaluated *once*, after development. Kaggle uses hidden games. The LLM-agent proxy uses all 25 public games (that agent has no training split). **Protocol** (`rl/evaluate.py`, identical for all agents). Each game gets 500 actions, counted across deaths, and play stops early only on WIN. Every run logs levels, actions, deaths, level-vs-action curves, recordings and the *local*

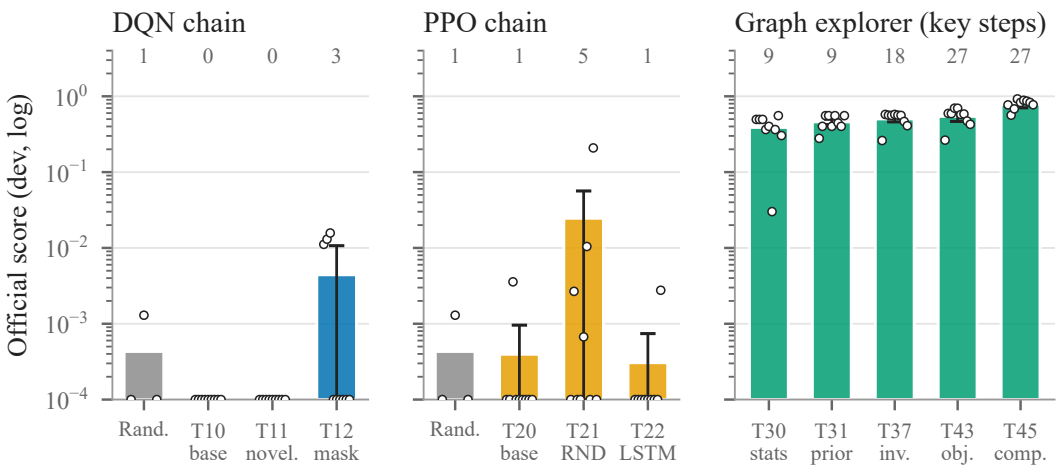


Figure 1: Local official score on dev (log scale; numbers = levels over all runs). Bars: mean over 3 training seeds, error bars: SD over training seeds; dots: all 9 runs, zeros drawn at 10^{-4}. Random/T30: no training (3/9 eval seeds). Colour = agent family throughout.

official score. Learned agents use 3 training $\times$ 3 evaluation seeds, and we report the mean $\pm$ SD over training seeds plus all runs. Training-free T30/T35n/T45n use 9 evaluation seeds. Every learned agent gets 400k steps with no early stopping, and the final checkpoint is used. The baselines, the evaluation-time convention and the Kaggle checkpoint (training seed 0) were pre-registered. **Hardware and cost.** Training ran on Kaggle T4 GPUs (Python 3.13.15, PyTorch 2.11.0+cu128, NumPy 2.1.3, SB3/sb3-contrib 2.9.0, Gymnasium 1.2.0 (Towers et al., 2024; Raffin et al., 2021)): 0.30–0.50 T4-hours per seed for DQN, 0.37–0.46 for PPO, 0.94 for PPO-LSTM, 11.90 in total over 24 runs. Evaluation ran on Kaggle CPU kernels at 1 thread per job. Per-action play time (agent plus environment step, excluding process setup) is 2–3 ms for random, 3–22 ms for the graph explorer, 6–9 ms for PPO and frozen DQN, and 140–184 ms for DQN-online because of its test-time TD updates (Table 2, supplement). The LLM agent needs an RTX PRO 6000 for 9 h.

4 Learning agents: the bottleneck is exploration inside an unseen game

All learned agents share one encoder: strided convolutions to an 8×8 map aligned with the click cells, a 1×1-convolution click head and an MLP head (1.19M parameters for DQN). They train on the 15 games as lock-step environments (hyperparameters: Table 3, supplement). **Baselines.** T10 is DQN with masked targets (replay 100k, batch 32, Adam 10^{-4}, γ 0.99, Huber loss, ϵ 1.0$\rightarrow$0.05 over 200k steps, hard target sync every 10k steps). At evaluation (`dqn-online`) it takes one TD update per action on the game being played (ϵ=0.05), with weights reset per game, and the frozen network is the variant T10f. T20 is MaskablePPO (Raffin et al., 2021). *What is learned:* a masked actor π_θ and a critic V_ϕ. *When:* after every 128-step rollout of the 15 environments (1,920 transitions), we compute GAE (γ 0.99, λ 0.95) and take 4 epochs of minibatch-240 updates on L^{CLIP} (ϵ=0.2), the value loss and an entropy bonus of 0.01. *Decisions:* at evaluation, actions are sampled from the frozen masked policy; within 500 actions an on-policy update would be only a few small batches. *Result:* both baselines are indistinguishable from random on dev (random 0.0004, 1 level in 3 runs; T10 and T10f 0; T20 0.0004, 1 level in 9). Fig. 2 shows why. The DQN seeds see only 7/4/8 rewards in 400k steps, so Q stays flat ($\approx$0.05) and greedy selection degenerates: T10f spends 77–93% of its actions on one action. PPO entropy only falls from 3.49 to $\approx$3.0 nats. *The limit is the missing learning signal, not capacity.*

Improvements: each step removes one diagnosed failure (limitation $\rightarrow$ hypothesis $\rightarrow$ intervention $\rightarrow$ result). *T11 (count novelty $\beta/\sqrt{N(s')}$, β=0.05).* A dense bonus should break the collapse. The training frame-change rate rises from 0.61 to 0.67, but dev stays at 0, and a new diagnostic

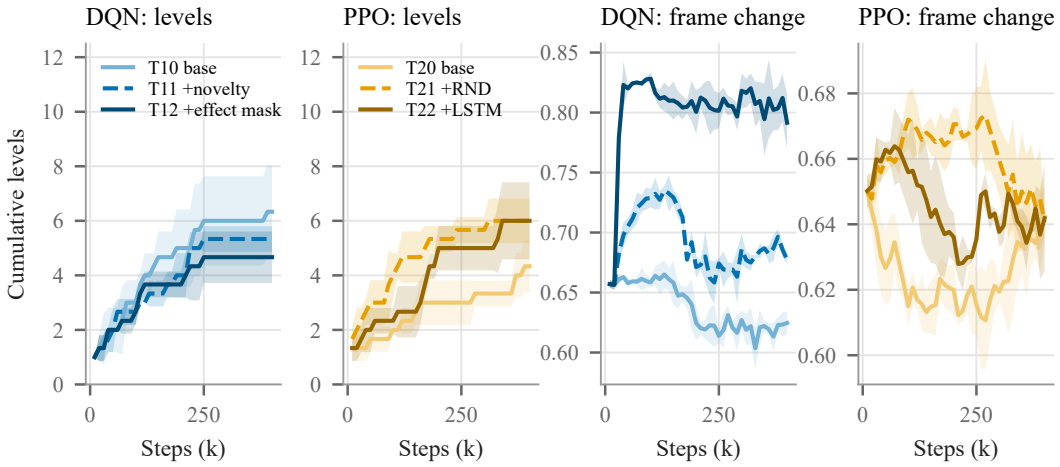


Figure 2: Training curves (mean ± SD over 3 training seeds). Left: cumulative level completions, i.e. every reward seen in 400k steps on 15 games. Right: fraction of actions that change the frame. No run sees more than 8 rewards.

shows that 27–33% of *all* actions repeat an (s, a) already seen to change nothing (random: 7%). *T12 (action-effect masking).* We add a head trained with BCE on "frame changed", mask actions with $p < 0.1$, and keep an exact per-game no-op memory at evaluation. Repeated no-ops fall to 0.169/0.012/0.042, and T12 is the first DQN to complete levels (0.0044 ± 0.0063, 3 levels). But the pretrained head is at *chance* on unseen games (within-game AUC 0.42–0.58), and so is a PPO-trained effect model (T23: 0.39–0.54). The gain comes from the per-game memory. *Ablation* (T12a, the mask without novelty, 3 new training seeds, same protocol): 0.0028 ± 0.0021 with 5 levels, and repeated no-ops of 0.083/0.004/0.099. Online, the mask alone does what T12 does (within noise), so masking is the active ingredient. Frozen, the mask-only policy completes 0 levels against 6 for T12f, so novelty shapes the pretrained policy without helping the online agent. *T21 (RND, coefficient 0.05).* A learned novelty signal should generalise across near-identical frames. RND first collapsed to 10^{-4}, because the frames are mostly background. Standardised target features and an EMA-normalised bonus fixed it. Result: the largest learned gain, 0.0247 ± 0.0317 and 5 levels, with high variance; a single ls20 solve in 41 actions scores 0.2085 by itself. *T22 (LSTM 256 on T21).* The memoryless T21 still repeats known no-ops in 8–10% of actions, and recurrence should let it remember failures. **It fails:** 0.0003 with 1 level, no fewer repeats (0.059–0.138), and twice the training cost. **Transfer.** On the held-out games all learned agents score 0, except T21, which scores 0.0005 (1 level in 9 runs). On its own training games, however, T21 scores 0.240: PPO+RND fits its training games and does not transfer. *Conclusion:* for both algorithms the binding constraint is exploration inside an unseen game. The knowledge that helps (what does nothing, where the agent has been) is specific to the game, so it must be explicit and acquired online. This motivates the next step.

5 Making the knowledge explicit: a graph explorer, and dev overfitting

Following *Stochastic Goose*, the explorer (`rl/policies.py`) keeps an exact frame-hash state graph per level and never re-tries recorded no-op or deadly (s, a) pairs. It walks a BFS path to the nearest state with untried actions and ranks them by per-game Beta-Thompson (Thompson, 1933) effect estimates, keyed by action or by the colour under the click; search with return-to-frontier relates to Go-Explore (Ecoffet et al., 2021). T30 uses no weights. T31 adds $0.3\,\pi(a|s)/\max\pi$ from T21 as a prior. Each later step fixed one failure seen in the recordings (supplement, Fig. 7). Masking the step-counter ("ticker") rows restores state identity (T32). Inverse action pairs learned online solve ls20 level 1 in 9/9 runs after 128–287 actions, instead of 6/9 after 367–473 (T37, 0.504). With 1000 or 2000 actions T37 reaches the same 18 levels, because the reachable frontier

Table 1: Master results. Local official scores (0–100 scale; our harness, offline) on train/dev/held-out, plus the Kaggle public leaderboard (hidden games, a single run). Dev: mean ± SD over training seeds; for the training-free T30, T35n and T45n, SD over 9 evaluation seeds (random: 3). Lv: levels over all dev runs (of 342; random of 114). Level 1: solving runs @ actions taken; no agent finished a game, so every run used all 500 actions. Train: 3 eval seeds (learned agents: training seed 0, on their own training games). Held-out: same runs as dev, evaluated once. –: not run or not submitted. †Starter random agent.

Ver.	Change	Dev score	Lv	Level 1 on dev: runs @ actions	Train	Held-out	Kaggle
Rand.		0.0004 ± 0.0006	1	lp85 1/3 @352	0.0111	0	0.02†
T10	DQN	0	0	–	–	0	0.00
T11	+novelty	0	0	–	–	–	0.00
T12	+effect mask	0.0044 ± 0.0063	3	lp85 3/9 @101–120	0.0002	0	0.00
T12a	mask only (no novelty)	0.0028 ± 0.0021	5	lp85 5/9 @146–421	–	–	–
T20	PPO	0.0004 ± 0.0006	1	lp85 1/9 @212	–	0	0.00
T21	+RND	0.0247 ± 0.0317	5	lp85 4/9 @124–489; ls20 1/9 @41	0.240	0.0005	0.05
T22	+LSTM	0.0003 ± 0.0004	1	lp85 1/9 @241	–	0	–
T30	graph+online stats	0.389 ± 0.149	9	lp85 9/9 @16–72	0.0578	–	–
T31	+PPO prior	0.461 ± 0.038	9	lp85 9/9 @12–24	–	–	–
T37	+inverse actions	0.504 ± 0.045	18	lp85 9/9 @15–25; ls20 9/9 @128–287	0.0305	0.133	0.07
T43	objects as fallback	0.545 ± 0.079	27	lf52 9/9 @52–349; lp85 9/9 @15–25; ls20 9/9 @128–287	–	–	–
T45	component clicks	0.791 ± 0.087	27	lf52 9/9 @24–56; lp85 9/9 @14–30; ls20 9/9 @128–287	0.0203	0.885	–
T45n	no prior	0.667 ± 0.222	28	lf52 9/9 @23–59; lp85 9/9 @11–33; ls20 8/9 @128–374	0.0811	0.767	–

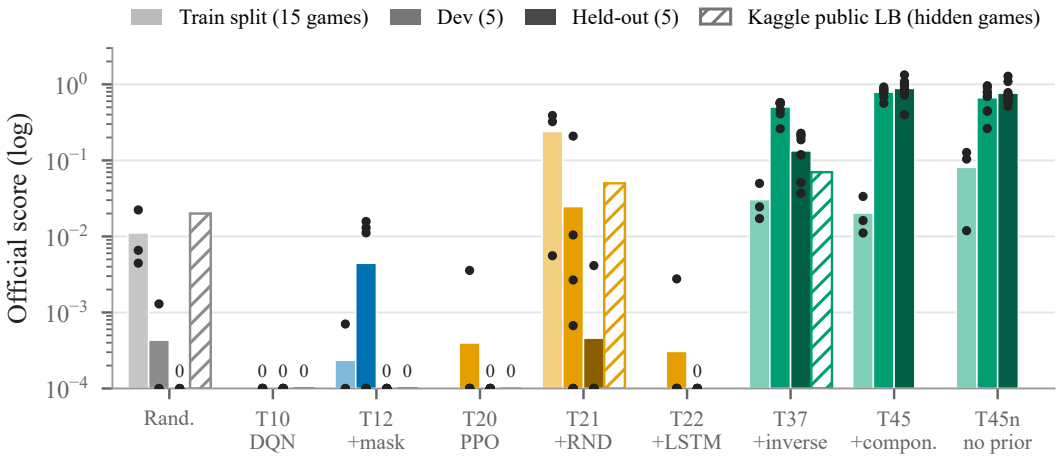


Figure 3: Local official score on each split, plus the Kaggle public score (log scale; dots = runs; "0" = scored zero; blank = not run or not submitted). Train: training seed 0, so for learned agents these are their own training games. Held-out: evaluated once.

under the 8×8 click grid is exhausted. With pixel clicks, one click per connected component (T45) gives **0.791 ± 0.087**, with lp85 level 1 in 14–30 actions (human 17) and lf52 in 24–56 (human 32). Some variants regressed (patch-keyed statistics T38: 0.124; size prior T47: 0.632). **Prior ablation:** without the PPO prior, T45n scores 0.667; the difference comes from ls20 (8/9 vs. 9/9 solves) and is within one SD of T45n's runs (0.222). The explorer's structure, not learning, does the work.

Generalisation (Fig. 3). The explorer's dev score was optimised over 18 versions on 5 games. Off dev, it holds up only where the dev fixes happen to apply. T37 falls from 0.504 on dev to 0.133 on held-out (22 levels: tu93 two levels in 9/9 runs, cd82 level 1 in 4/9) and **0.07** on Kaggle. T45 scores 0.885 on held-out (34 levels). About three quarters of that, by our per-game RHAE reconstruction, comes from one game, tn36: level 1 in 22–45 actions against a human baseline of 32. bp35 and sk48 are never solved. On the 15 train games, which the explorer's logic never saw, T45 scores only 0.020 (11 levels) and T45n 0.081. *Dev overfitting is real but game-dependent:* component clicks

transfer to click-driven games, while orderings tuned to five games do not, and no version reaches the late levels that RHAE weights most.

Final agents compared. *T12 (final DQN):* masking cuts wasted actions, but held-out is 0 and even its own training games score 0.0002. It is the slowest agent (0.5 h training per seed, 167–178 ms/action at evaluation for test-time TD). *T22 (final PPO)* fails on every split (0.94 h/seed). *T21 (best PPO)* learns behaviour specific to its training games (0.240 there), not transferable exploration (held-out 0.0005, Kaggle 0.05), at 0.46 h/seed and 6–7 ms/action. *T45 (graph)* needs no training beyond the T21 prior and adapts online through explicit memory. It is the only agent that completes levels on unseen games, at or near human speed where it succeeds (lp85, lf52, tn36), at 12–22 ms/action on one CPU thread. Its weakness is shared with the learned agents: it never finishes a game, and it stalls once its frontier is exhausted. Across all 25 public games T45 averages about 0.35. Franzen's public LLM agent scores 46.5 on the same games (skarin, 2026), two orders of magnitude more. This gap is why we turned to the state of the art.

6 Research phase: the state of the art

We studied (i) the competition's milestone winners (Stochastic Goose, Duck, Franzen M2); (ii) 26 public notebooks (the top by score and the most-voted on 16 Oct, plus those run 12–17 Oct), each summarised and diffed against its parent; (iii) the harness literature (OpenAI, 2026); and (iv) the leaderboard. Under our integrity rules we *excluded* a byte-identical copy with its credits stripped, two notebooks with hard-coded public game IDs or opaque encoded patches, and a BFS solver that reads game source and hidden state. Nearly every strong public notebook derives from Franzen's M2. The two leaders (68.91 and 67.49 on 16 Oct) publish no code. Copies of Franzen's notebook score 25.77 ± 3.93 on the public leaderboard. We therefore adopted Franzen's M2 (Franzen, 2026) on Tufa Labs' harness (Cottaar & Tufa Labs, 2026) as a *research platform*: the strongest reproducible, documented and licensed agent, used to test whether our findings (stalls, wasted actions) also limit a strong pretrained agent. The agent itself is not our work.

7 Our work on the platform

Reproducible fork and proxy (ours). We pinned the Docker image (the 1 Oct image update broke the bundled Python 3.12 wheels) and generate every variant with a script from the public notebooks. In the submission packaging, heavy cells run only in the competition rerun, Save & Run writes a placeholder, and development versions have a 3-game, 5-minute smoke path. For the unchanged fork, static checks confirm that the competition code path is byte-identical to the original, and it scores 28.83 on Kaggle (Franzen: 27.89). Our proxy plays all 25 public games for one pass with the same per-game compute share as the competition, taking $\sim$2 h on an RTX PRO 6000; at 6 GPU-hours per week we could afford two at most.

Where the points go (ours). On the 100 public trajectories of Franzen's v3 run (skarin, 2026), our script reproduces every stored score. Stalls on later levels cost 43.9 points, level 1 never solved 7.0, and inefficiency on solved levels only 2.6; agents take a median of $0.64\times$ the human actions on the levels they solve. Our own MOD-1 proxy logs (2,341 model replies) agree: 50.2 points are lost to unfinished levels and 2.46 to excess actions. We then ran eight independent, read-only loss searches over our logs and the harness code (scoring, scheduling, serving, prompts, context, game behaviour, harness bugs, rules). These were LLM-assisted and every claim was re-checked by us against the code and logs. They found no single large leak, only ranked small ones. *The LLM agent, like ours, is efficient where it knows what to do and loses points where it runs out of hypotheses.*

Hypothesis tests. (a) *Retained reasoning* (OpenAI, 2026) is already active: prior thinking is resent on every assistant turn (prompt growth per generated token: median 1.83, never below 0.5, although 87.5% of generated tokens are reasoning; prefix-cache hit rate 93%), and Franzen found no gain from compaction, so we built nothing. (b) *MOD-1*, two reused add-ons (heliosli, 2026; shiiin9 & Affectify ARC, Osaka Metropolitan University, 2026), scores 47.39 on the proxy, inside

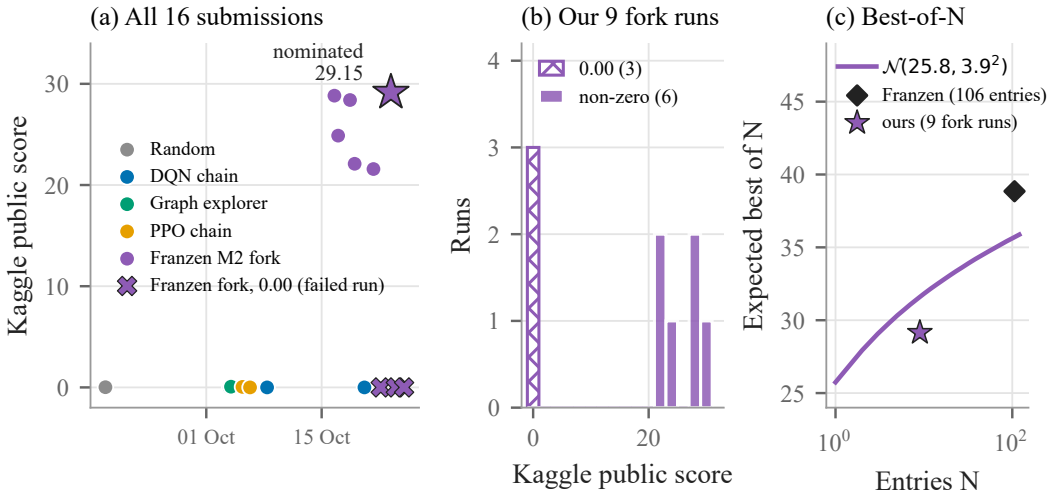


Figure 4: Kaggle public scores (hidden games). (a) All 16 of our submissions; $\times$: fork runs scoring 0.00 (byte-identical 3,411 B outputs); $\star$: nominated. (b) Our 9 fork runs. (c) Expected best of N submissions under the spread of public copies of the notebook, with Franzen's best (38.85 from 106 entries, 2026-10-25) and ours.

Franzen's per-pass range of 45.6–47.5, and 24.88 and 21.58 on Kaggle: no evidence of a gain. (c) *MOD-2* targets stalls. Tokens spent past 62k on a level return about half the average points per token, so our **reinvestment rule** decays a game's scheduling priority after 130k tokens on one level (factor $\max(0.2, e^{-(t-130\text{k})/100\text{k}})$), alongside reused Fable facts (leoprovorov, 2026). Against a pre-registered threshold of +2 in a matched A/B test (launched simultaneously), MOD-2 scored -4.56 (95% bootstrap CI -15.2 to $+5.4$; 97 vs. 107 levels; supplement, Fig. 8). Post-mortem: seven games stopped acting 18–69 min into the run while still alive, starved by our rule, and five of them explain $\approx$7.8 points. We did not submit it. (d) **MOD-F**, our three fixes, each from the audit: (1) reasoning-only replies cut off at the 12,288-token cap were re-sent verbatim (13 replies, $\sim$13k tokens each), and we enable the harness's existing stub; (2) a refusal message falsely said a stale-state error "can be caught" (92 actions silently refused), and we correct it and add one prompt line; (3) we cut the per-game cap from 532 to 525 min, because proxy timings projected 32,524 s against the 32,400 s limit. All changes and evidence are in `MODIFICATIONS.md`.

What a Kaggle score can show (ours). MOD-F scored 28.40, 22.10, 0.00, **29.15**, 0.00 and 0.00 (Fig. 4). We nominate submission 57556604 (29.15, 2026-10-23 09:42:37 UTC; rank 959 of 4,817 on 25 Oct). A leaderboard entry is a team's *best* submission. Under the copies' spread, the expected best of N grows from 25.8 at $N{=}1$ to $\approx$35.6 at $N{=}100$, and Franzen's 38.85 from 106 entries is consistent with this plus genuine gains. All our non-zero runs (21.58–29.15) lie inside the spread, so *we cannot claim that any modification changes the expected score*; one proxy pass resolves only effects of about $\pm$10 (per-game SD $\approx$26). **Zero runs:** three MOD-F runs produced byte-identical outputs in which no game scored, i.e. whole-run failures. Our changes are deterministic, the same version scored three times, and the inputs were unchanged. If zeros fell at random among the 9 fork runs, all 3 would land in MOD-F's 6 with probability 0.24. We find no evidence of a code cause, but the rerun logs are not visible to us.

8 Failure cases and discussion

One bottleneck, every agent. On move and mixed dev games every agent dies on a fixed per-life step budget ($\sim$129/125/100/65 steps on ls20/dc22/ka59/lf52), identical for random, PPO and graph agents, so banning the action before a death is wrong (T34). On dc22 the reachable states are exhausted (76 after 500 actions, Fig. 5), and relaxing pruning (T40) finds none new. What is missing

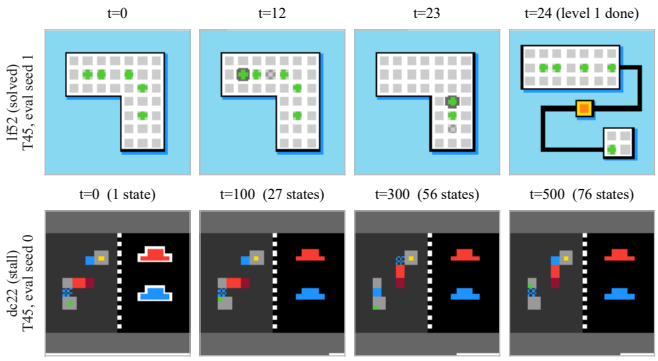


Figure 5: Recorded T45 frames (local run). Top: lf52 level 1 solved in 24 actions (human 32); the next layout appears at t=24. Bottom: dc22 stall, with 76 distinct ticker-masked states after 500 actions.

is a hypothesis about the goal, not more coverage. Measured in points, the same holds for the LLM agent: 43.9 points are lost to later-level stalls but only 2.6 to inefficiency. **Established.** (1) With sparse level rewards and 15 training games, DQN and PPO do not learn transferable exploration; count, RND, effect and recurrence changes do not fix this. (2) Pretrained effect predictors are at chance on unseen games. (3) Explicit per-game memory transfers where its mechanisms apply (tn36, at human speed) and fails elsewhere (0.02 on the train games). (4) Single Kaggle scores cannot rank small modifications of an LLM agent. **Uncertain.** Graph versions within ≈0.05 of each other are within seed noise; dev and held-out have 5 games each; Kaggle scores are single runs with unexplained failures. **Limitations.** 400k steps is small for deep RL (Henderson et al., 2018), although only 3–8 rewards per run suggests that the signal, not the budget, binds. Only the graph explorer (from T41) may click any pixel, so its action space is larger than the learned agents' `Discrete(70)`, and graph-vs-RL comparisons from T41 on are not matched in action space. Compute is not matched between our agents (CPU seconds) and the LLM agent (9 GPU-hours). Our loss searches were LLM-assisted.

9 Conclusion

RL agents trained on public ARC-AGI-3 games did not learn to explore unseen games. Every gain came from making per-game knowledge explicit and online, and those gains transferred only where their mechanisms applied. A strong pretrained LLM agent shares the same bottleneck: it is efficient on the levels it understands and stalls on later levels. Inferring goals, rather than covering states faster, is where both learned and pretrained agents must improve.

Generative-AI declaration. Per the Wits academic-integrity policy: we used Claude and Claude Code (Anthropic) for planning, code, data analysis (including the LLM-assisted loss searches) and drafting. We verified all code, numbers and text against our logs and take full responsibility for the content.

Member contributions. Both authors contributed equally: research design and protocol, implementation and experiments (run jointly on one machine), analysis, and writing.

References

ARC Prize Foundation. ARC-AGI-3: Interactive reasoning benchmark. https://arcprize.org/arc-agi/3, 2026. Accessed 2026-10-25.

ARC Prize Foundation and Kaggle. ARC Prize 2026 – ARC-AGI-3. https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-3, 2026. Kaggle code competition; public leaderboard snapshot 2026-10-25.

Marc G. Bellemare, Sriram Srinivasan, Georg Ostrovski, Tom Schaul, David Saxton, and Rémi Munos. Unifying count-based exploration and intrinsic motivation. In *Advances in Neural Information Processing Systems*, 2016.

Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation. In *International Conference on Learning Representations*, 2019.

François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

Jeroen Cottaar and Tufa Labs. The duck: ARC-AGI-3 inference harness (milestone 1 winning solution). https://github.com/Tufalabs/duck-harness, 2026. Commit 7652836; Kaggle notebook jeroencottaar/tufa-labs-duck-harness-june-30-milestone-winner.

Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O. Stanley, and Jeff Clune. First return, then explore. *Nature*, 590(7847):580–586, 2021.

Daniel Franzen. ARC-AGI-3 milestone 2 solution. https://www.kaggle.com/code/dfranzen/arc-agi-3-milestone-2-solution, 2026. Code: https://github.com/da-fr/arc-agi-3-solution (Apache-2.0); write-up: https://www.kaggle.com/writeups/dfranzen/arc-agi-3-milestone-2.

heliosli. Try merging it, you might get lucky and win gold (stagnation restart). https://www.kaggle.com/code/heliosli/try-merging-it-you-might-get-lucky-and-win-gold, 2026. Kaggle notebook, V4.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

Shengyi Huang and Santiago Ontañón. A closer look at invalid action masking in policy gradient algorithms. In *Proceedings of the International FLAIRS Conference*, volume 35, 2022.

leoprovorov. Fable / ASTRA play: ARC-3 handbook harness (F001 fable layer). https://www.kaggle.com/code/leoprovorov/fable-astra-play-arc-3-handbook-harness, 2026. Kaggle notebook and dataset, Apache-2.0.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

OpenAI. How two settings tripled our ARC-AGI-3 scores. https://openai.com/index/how-two-settings-tripled-our-arc-agi-3-scores/, 2026. 29 July 2026.

Qwen Team. Qwen3.8-Flash-Next, 2026. Used as Intel AutoRound W4A16 quantisation (Kaggle model dfranzen/intel-qwen3.8-flash-next-w4a16-autoround) with Albucino's MTP draft model; Qwen Community License 1.0.

Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

shiiin9 and Affectify ARC, Osaka Metropolitan University. Affectify ARC 31.54 in a single sub (D′ slot priority). https://www.kaggle.com/code/shiiin9/affectify-arc-31-54-in-a-single-sub, 2026. Kaggle notebook.

skarin. ARC-AGI-3 27.80 LB – 100-trajectory audit. https://www.kaggle.com/code/skarin/arc-agi-3-27-80-lb-100-trajectory-audit, 2026. Kaggle notebook; source of the 100 Franzen v3 trajectories we re-analysed.

Dries Smit. Stochastic goose: ARC-AGI-3 developer preview agent competition solution. https://github.com/DriesSmit/ARC3-solution, 2025. Tufa Labs.

William R. Thompson. On the likelihood that one unknown probability exceeds another in view of the evidence of two samples. *Biometrika*, 25(3/4):285–294, 1933.

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, 2024.

Supplementary Materials

The following content was not necessarily subject to peer review.

10 Hyperparameters, compute and all versions

Table 2: Compute. Training on one Kaggle T4 per seed (seeds in parallel). Evaluation on Kaggle CPU at 1 thread per job. "Dev eval wall s" is the summed wall time of the evaluation processes for all dev runs (runs × 5 games × 500 actions), including setup and model loading. "Play ms/action" is the agent plus environment step only, from per-game timers. Test-time TD learning makes DQN-online ∼20× slower per action than PPO. Total training: 11.90 T4-hours over 24 runs (incl. the T23 effect-model diagnostic and the T12a ablation).

Version	Train steps	Train h/seed (T4)	Train lv (s0/s1/s2)	Dev eval wall s (all runs)	Play ms/action (dev / held-out)
T10 DQN	400,005	0.30–0.32	7/4/8	3,499 (9 runs)	153.5 / 149.7
T11 +novelty	400,005	0.43–0.43	5/5/6	3,198 (9 runs)	140.3
T12 +effect mask	400,005	0.49–0.50	4/6/4	4,050 (9 runs)	177.9 / 166.5
T12a mask only (no novelty)	400,005	0.48–0.49	3/6/3	4,193 (9 runs)	184.0
T20 PPO	401,280	0.37–0.37	3/5/5	329 (9 runs)	8.0 / 6.8
T21 +RND	401,280	0.45–0.46	5/7/6	203 (9 runs)	6.0 / 6.6
T22 +LSTM	401,280	0.93–0.94	5/5/8	326 (9 runs)	8.9 / 8.2
Random	0	–	–	23 (3 runs)	2.8 / 1.7
T30 graph+online stats	0	–	–	89 (9 runs)	3.6
T37 +inverse actions	0 (+T21 prior)	–	–	277 (9 runs)	6.9 / 10.6
T45 component clicks	0 (+T21 prior)	–	–	369 (9 runs)	11.8 / 22.3
T45n no prior	0	–	–	138 (9 runs)	5.9 / 16.1

Table 3: Hyperparameters (identical across seeds; all learned versions: 400k steps on 15 games, episode cap 500; configs in `experiments/train/*/config.json`).

DQN (T10–T12)	replay 100k, batch 32, Adam 10^{-4}, γ 0.99, Huber, train every 4 steps, start 20k, hard target sync 10k, ϵ 1.0→0.05 over 200k; eval: ϵ 0.05, 1 TD update/action (batch 32, lr 10^{-4}, target sync 100)
T11 / T12	novelty $\beta = 0.05$ (exact frame counts) / effect head BCE weight 1.0, mask threshold $p < 0.1$
PPO (T20–T22)	MaskablePPO, n_{steps} 128 × 15 envs, batch 240, 4 epochs, Adam $2.5 \cdot 10^{-4}$, γ 0.99, GAE λ 0.95, clip 0.2, entropy 0.01
T21 / T22	RND coefficient 0.05, EMA(α=0.01)-normalised / + LSTM 256 (RecurrentPPO, masks as input channel)
Graph (T30–T47)	Beta(1,1)-Thompson effect stats; prior weight 0.3 (T31 +); prune after 3 failed tries; inverse pair after 2 undos, 0 counter-ex.; ticker row: ≥10 changing steps, changes in ≥50%, median ≤2 px, ≥3 actions; components ≤96 per state

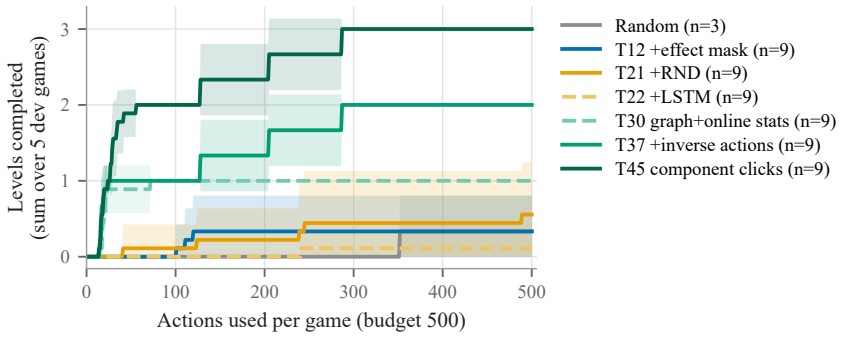


Figure 6: Levels completed on the 5 dev games vs. actions used (local, mean ± SD over runs).

11 Platform experiments and Kaggle evidence

All 16 Kaggle submissions with IDs, script versions and output sizes are in `experiments/kaggle_submissions.csv`. Nominated: submission **57556604**, notebook `thendonelufule/arc3-m2-modf-sub`, script version 359469291, public score **29.15**, 2026-10-23 09:42:37 UTC.

Table 4: Every evaluated version (same columns as Table 1). G1 = graph exploration with the pretrained T12 effect head.

Ver.	Change	Dev score	Lv	Level 1 on dev: runs @ actions	Train	Held-out	Kaggle
Rand.		0.0004 ± 0.0006	1	lp85 1/3 @352	0.0111	0	0.02†
T10	DQN	0	0	–	–	0	0.00
T10f	DQN frozen	0	0	–	–	–	–
T11	+novelty	0	0	–	–	–	0.00
T11f	frozen	0	0	–	–	–	–
T12	+effect mask	0.0044 ± 0.0063	3	lp85 3/9 @101–120	0.0002	0	0.00
T12f	frozen	0.0036 ± 0.0033	6	lp85 6/9 @116–307	–	–	–
T12a	mask only (no novelty)	0.0028 ± 0.0021	5	lp85 5/9 @146–421	–	–	–
T12af	mask only, frozen	0	0	–	–	–	–
T20	PPO	0.0004 ± 0.0006	1	lp85 1/9 @212	–	0	0.00
T21	+RND	0.0247 ± 0.0317	5	lp85 4/9 @124–489; ls20 1/9 @41	0.240	0.0005	0.05
T22	+LSTM	0.0003 ± 0.0004	1	lp85 1/9 @241	–	0	–
G1	graph + T12 effect head	0.0013 ± 0.0010	4	lp85 4/9 @149–380	–	–	–
T30	graph+online stats	0.389 ± 0.149	9	lp85 9/9 @16–72	0.0578	–	–
T31	+PPO prior	0.461 ± 0.038	9	lp85 9/9 @12–24	–	–	–
T32	+ticker mask	0.462 ± 0.038	11	lp85 9/9 @12–24; ls20 2/9 @413–416	–	–	–
T33	+prune	0.484 ± 0.037	9	lp85 9/9 @12–20	–	–	–
T34	+timer deaths	0.484 ± 0.037	9	lp85 9/9 @12–20	–	–	–
T35	+click equivalence	0.493 ± 0.045	9	lp85 9/9 @15–25	–	–	–
T35n	no prior	0.471 ± 0.073	15	lp85 9/9 @15–21; ls20 6/9 @297–473	0.121	–	–
T36	prior on clicks	0.494 ± 0.045	15	lp85 9/9 @15–25; ls20 6/9 @367–473	–	–	–
T37	+inverse actions	0.504 ± 0.045	18	lp85 9/9 @15–25; ls20 9/9 @128–287	0.0305	0.133	0.07
T38	patch stats	0.124 ± 0.081	18	lp85 9/9 @17–220; ls20 9/9 @128–376	–	–	–
T39	value frontier	0.497 ± 0.045	18	lp85 9/9 @15–25; ls20 9/9 @205–421	–	–	–
T40	relax when stuck	0.503 ± 0.045	18	lp85 9/9 @15–25; ls20 9/9 @128–458	–	–	–
T41	+object clicks	0.470 ± 0.088	26	lf52 9/9 @34–95; lp85 9/9 @17–36; ls20 6/9 @240–413	–	–	–
T42	small objects	0.379 ± 0.094	28	ka59 1/9 @256; lf52 9/9 @35–99; lp85 9/9 @17–33; ls20 9/9 @128–287	–	–	–
T43	objects as fallback	0.545 ± 0.079	27	lf52 9/9 @52–349; lp85 9/9 @15–25; ls20 9/9 @128–287	–	–	–
T44	object promotion	0.568 ± 0.063	27	lf52 9/9 @46–295; lp85 9/9 @15–25; ls20 9/9 @128–287	–	–	–
T45	component clicks	0.791 ± 0.087	27	lf52 9/9 @24–56; lp85 9/9 @14–30; ls20 9/9 @128–287	0.0203	0.885	–
T45n	no prior	0.667 ± 0.222	28	lf52 9/9 @23–59; lp85 9/9 @11–33; ls20 8/9 @128–374	0.0811	0.767	–
T46	centres fallback	0.791 ± 0.087	27	lf52 9/9 @24–56; lp85 9/9 @14–30; ls20 9/9 @128–287	–	–	–
T47	size prior	0.632 ± 0.077	27	lf52 9/9 @13–45; lp85 9/9 @16–49; ls20 9/9 @128–287	–	–	–

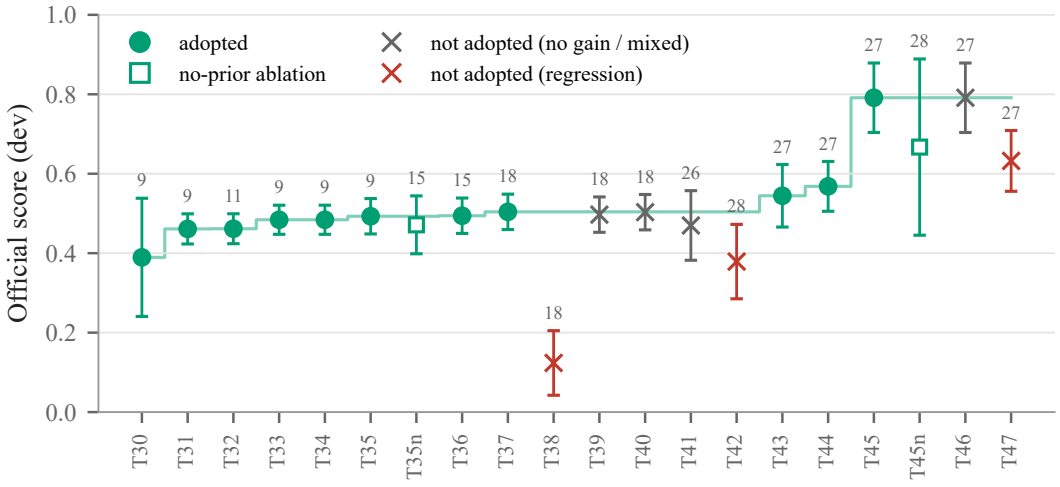


Figure 7: Graph-explorer progression on dev (local; mean ± SD over training seeds; numbers = levels). The line is the best adopted version so far. Red = regressions (T38 patch-keyed stats, T42 small objects only, T47 size prior).

Table 5: Every experiment on the LLM-agent platform. Proxy = local score on the 25 public games (1 pass); Kaggle = public leaderboard (hidden games).

Version	What	Authorship	Result
Base	Franzen M2, unchanged agent; our packaging + smoke path	reused (packaging ours)	Kaggle 28.83
MOD-1	+ stagnation restart (heliosli, 2026) + D′ scheduler (shiiin9 & Affectify ARC, Osaka Metropolitan University, 2026)	reused add-ons	proxy 47.39; Kaggle 24.88, 21.58
MOD-2	MOD-1 + Fable facts (leoprovorov, 2026) + our reinvestment rule	rule ours	proxy 42.83 (−4.56); not submitted
MOD-F	base + 3 fixes (stub, refusal message + prompt line, 525-min cap)	ours	Kaggle 28.40, 22.10, 0, 29.15, 0, 0
Retention	check of retained reasoning in logs and code	ours	already active; nothing built

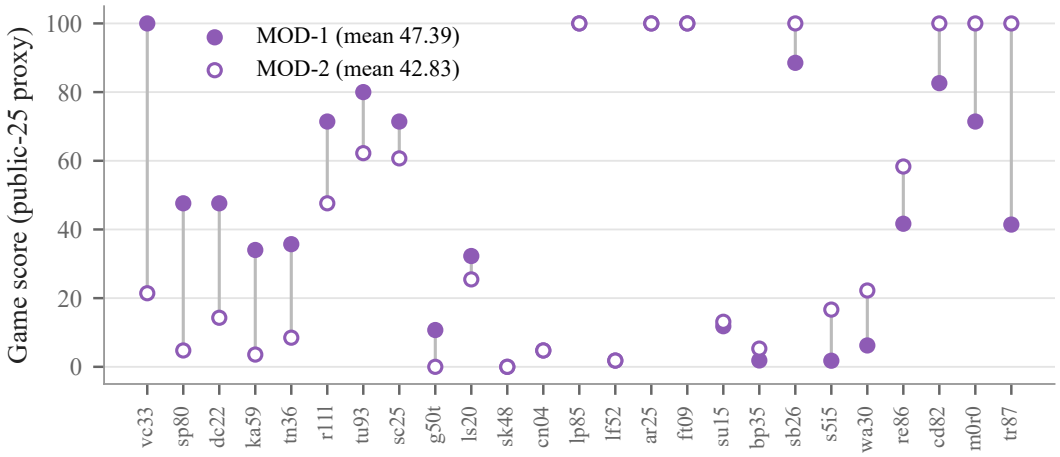


Figure 8: Matched one-pass proxy on the 25 public games (local scores of the LLM agent; RTX PRO 6000, ≈2 h each, launched together): MOD-1 47.39 vs. MOD-2 42.83. Per-game swings of up to 79 points in both directions show why one pass resolves only large effects.